22360859088
KORAY GARİP

BURSA TEKNİK ÜNİVERSİTESİ

MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

Node.JS ile Web Programlama

LAB_6

1. API Kullanımı ve Kurulum

Bu aşamada yeni bir uygulamaya geçilerek hava durumu verisi çekme üzerine çalışıldı. Bunun için weatherstack.com üzerinden bir API key alındı ve projeye eklendi. HTTP istekleri göndermek için npm i postman-request komutu ile postman-request kütüphanesi kuruldu.

```
weather > JS app.js > request() callback
const request=require("postman-request");
const url="http://api.weatherstack.com/current?access_key=2f555b90f43de6969ac39210ee6559a9&query=Mudanya"



request({url:url,json:true},(err,res,body)=>{
console.log(body.current)})
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\koray\Desktop\nodeproject\weather> node .\app.js
    co: '154.85',
    no2: '3.05',
    o3: '96',
    so2: '4.05',
    pm2_5: '11.55',
    pm10: '13.25',
    'us-epa-index': '1',
    'gb-defra-index': '1'
  },
  wind_speed: 9,
  wind_degree: 10,
  wind_dir: 'N',
  pressure: 1020,
  precip: 0,
  humidity: 38,
  cloudcover: 50,
  feelslike: 14,
  uv_index: 1,
  visibility: 10,
  is_day: 'yes'
}
PS C:\Users\koray\Desktop\nodeproject\weather>
```

2. API'den Veri Çekme

API'ye istek atılarak hava durumu bilgileri alındı. Gelen verilerden:

- sıcaklık (temperature)
- hissedilen sıcaklık (feelslike)
- yağış miktarı (precip)
- hava durumu açıklaması (weather_descriptions)

gibi bilgiler alınarak kullanıcıya gösterildi.

Örnek kullanımda alınan verilere göre yağış durumu yorumlanarak ekrana yazdırıldı. Böylece API'den gelen ham verinin işlenip anlamlı çıktıya dönüştürülmesi sağlandı.

```
weather > JS app.js > ...

const request=require("postman-request");
const url="http://api.weatherstack.com/current?access_key=2f555b90f43de6969ac39210ee6559a9&query=Mudanya"



request({url:url,json:true},(err,res,body)=>{
    const temp=body.current.temperature
    const feelTemp=body.current.feelslike
    const precip=body.current.precip
    const weather=body.current.weather_descriptions[0];

    console.log(`Hava ${temp} derece ve hissedilen sicaklik ${feelTemp}`);
   if(precip===0) console.log("Yagis beklenmiyor");
   else if(precip <=10) console.log("Az Yagis bekleniyor");
   else if(precip<25) console.log("Ortalama yagis bekleniyor");
   else if(precip<50) console.log("Sağanak Yagis bekleniyor");
   else console.log("Fırtına bekleniyor");
});
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\koray\Desktop\nodeproject\weather> node .\app.js
Hava 13 derece ve hissedilen sicaklik 12
Yagis beklenmiyor
PS C:\Users\koray\Desktop\nodeproject\weather>
```

3. Geocoding (Konum Dönüştürme)

Başlangıçta koordinat (lat, lon) ile işlem yapılması planlandı. Bunun için geocoding API kullanımı ele alındı. Mapbox API ücretli olduğu için alternatif olarak:

https://geocoding-api.open-meteo.com/v1/search API'si kullanıldı.

Ayrıca URL içinde özel karakter sorunlarını önlemek için encodeURIComponent() veya request içinde qs (query string) kullanımı incelendi.

Örnek

```
weather > JS app.js > ...

const geourl="https://geocoding-api.open-meteo.com/v1/search"

const params = {
        name: "Los Angeles",
        count:1,
        language:"tr",
        format:"json"
    }


request({ url: geourl, qs: params, json: true }, (err, res, body) => {
    if (err) {
        console.error("Hata:", err);
        return;
    }
    console.log(body)
    console.log("**************************************************")
    const name=body.results[0].name
    const lat=body.results[0].latitude
    const lon=body.results[0].longitude
    console.log(`${name} için\nlat:${lat}\nlon:${lon}`)
});
```

```
TERMINAL
PS C:\Users\koray\Desktop\nodeproject\weather> node .\app.js
{
  results: [
    {
      id: 5368361,
      name: 'Los Angeles',
      latitude: 34.05223,
      longitude: -118.24368,
      elevation: 89,
      feature_code: 'PPLA2',
      country_code: 'US',
      admin1_id: 5332921,
      admin2_id: 5368381,
      timezone: 'America/Los_Angeles',
      population: 3820914,
      postcodes: [Array],
      country_id: 6252001,
      country: 'ABD',
      admin1: 'Kaliforniya',
      admin2: 'Los Angeles County'
    }
  ],
  generationtime_ms: 1.0757446
}
**************************************************
Los Angeles için
lat:34.05223
lon:-118.24368
PS C:\Users\koray\Desktop\nodeproject\weather>
```

```
const address = "Çanakkale";

const geourl = `https://geocoding-api.open-meteo.com/v1/search?name=${encodeURIComponent(address)}&
count=1&language=tr&format=json`;

request({ url: geourl, json: true }, (err, res, body) => {
    if (err) {
        console.error("Hata:", err);
        return;
    }
    console.log(body)
    console.log("**************************************************")
    const name=body.results[0].name
    const lat=body.results[0].latitude
    const lon=body.results[0].longitude
    console.log(`${name} için\nlat:${lat}\nlon:${lon}`)
});
```

Not olarak, güncel durumda weatherstack API'nin geocoding ile çalışmadan lat lon bilgisi verme yerine doğrudan şehir ismi ile çalışabildiği öğrenildi.

4. Hata Yönetimi

API kullanımı sırasında hata yönetiminin önemi vurgulandı. Request yapısında kullanılan:

(err, res, body)

parametreleri üzerinden hatalar kontrol edildi.

- Eğer err varsa → internet bağlantısı veya API isteği hatasıdır
- Eğer body.error varsa → API'den gelen veri hatalıdır

Bu nedenle kullanıcıya uygun hata mesajı verilmesi gerektiği öğrenildi.

Örnek

İlk olarak internet kapalı çalıştırılıp terminale çıktı alındı sonrasında urli hatalı girerek örneğin (/searchh) son kısımda ise doğru şekilde çalıştırılıp çıktıların hepsi tek terminalde toplandı.

```
weather > JS app.js > geourl

const geourl="https://geocoding-api.open-meteo.com/v1/search"

const params = {
        name:"Çanakkale",
        count:1,
        language:"tr",
        format:"json"
    }



request({ url: geourl, qs: params, json: true }, (err, res, body) => {
    if (err) {
        console.log("İnternet Hatası:", err.code);
        return;
    }
    else if(body.error){console.log("url bilgilerini kontrol etmelisin")}
    else{
        console.log(body)
        console.log("**************************************************")
        const name=body.results[0].name
        const lat=body.results[0].latitude
        const lon=body.results[0].longitude
        console.log(`${name} için\nlat:${lat}\nlon:${lon}`)
    }
});
```

```
TERMINAL
PS C:\Users\koray\Desktop\nodeproject\weat
her> node .\app.js
İnternet Hatası: ENOTFOUND
PS C:\Users\koray\Desktop\nodeproject\weat
her> node .\app.js
url bilgilerini kontrol etmelisin
PS C:\Users\koray\Desktop\nodeproject\weat
her> node .\app.js
{
  results: [
    {
      id: 749780,
      name: 'Çanakkale',
      latitude: 40.15552,
      longitude: 26.41271,
      elevation: 12,
      feature_code: 'PPLA',
      country_code: 'TR',
      admin1_id: 749778,
      timezone: 'Europe/Istanbul',
      population: 143622,
      country_id: 298795,
      country: 'Türkiye',
      admin1: 'Çanakkale'
    }
  ],
  generationtime_ms: 0.86688995
}
**************************************************
Çanakkale için
lat:40.15552
lon:26.41271
PS C:\Users\koray\Desktop\nodeproject\weat
her>
```